

Java Relational & Logical Operators in Real Conditions

Learning Objectives

After this topic, you will be able to:

- Understand relational operators.
 - Understand logical operators.
 - Combine multiple conditions using logical operators.
 - Write real-world conditional statements.
 - Solve common interview and output-based questions.
-

1. Relational Operators

Relational operators compare two values and always return a **boolean** (true or false).

Operator	Meaning	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

Example

```
int age = 22;  
  
System.out.println(age >= 18); // true  
System.out.println(age < 18);  // false
```

2. Logical Operators

Logical operators combine two or more boolean expressions.

Operator	Meaning	Example
&&	Logical AND	a > 5 && b < 10
,		,
!	Logical NOT	!isLoggedIn

3. Real-World Examples

Check Voting Eligibility

```
int age = 20;

if (age >= 18) {
    System.out.println("Eligible to vote");
}
```

Login Validation (AND)

```
boolean emailCorrect = true;
boolean passwordCorrect = true;

if (emailCorrect && passwordCorrect) {
    System.out.println("Login Successful");
}
```

Both conditions must be **true**.

Scholarship Eligibility

```
double cgpa = 3.7;
boolean sportsQuota = false;

if (cgpa >= 3.5 || sportsQuota) {
    System.out.println("Scholarship Eligible");
}
```

Only **one** condition needs to be true.

NOT Operator

```
boolean accountBlocked = false;

if (!accountBlocked) {
    System.out.println("Access Granted");
}
```

4. Short-Circuit Evaluation

Java uses **short-circuit evaluation** with && and ||.

AND (&&)

If the first condition is **false**, Java does **not** check the second.

```
if (false && (10 / 0 > 1)) {
    System.out.println("Hello");
}
```

No exception occurs because the second condition is never evaluated.

OR (||)

If the first condition is **true**, Java skips the second condition.

```
if (true || (10 / 0 > 1)) {  
    System.out.println("Hello");  
}
```

Output:

```
Hello
```

5. Common Mistakes

 Using = instead of ==

```
if (age = 18)    // Error
```

 Correct

```
if (age == 18)
```

 Comparing Strings with ==

```
String s1 = "Java";  
String s2 = new String("Java");  
  
System.out.println(s1 == s2); // false
```

 Correct

```
System.out.println(s1.equals(s2)); // true
```

6. Best Practices

- Use && when **all** conditions must be true.
 - Use || when **at least one** condition must be true.
 - Use ! to reverse a boolean condition.
 - Use equals() to compare String values, not ==.
-

7. Tricky Interview Questions

Q1. What is the difference between == and !=?

Answer:

- == checks equality.
 - != checks inequality.
-

Q2. What is the difference between && and ||?

Answer:

- && → All conditions must be true.
 - || → At least one condition must be true.
-

Q3. What does !true return?

Answer:

false

Q4. What is short-circuit evaluation?

Answer:

Java skips evaluating the remaining condition when the final result is already determined.

Q5. Which method should be used to compare String contents?

Answer:

equals()

8. Output-Based Questions

Question 1

```
System.out.println(10 > 5);
```

Output

```
true
```

Question 2

```
System.out.println(10 == 20);
```

Output

```
false
```

Question 3

```
System.out.println(true && false);
```

Output

```
false
```

Question 4

```
System.out.println(true || false);
```

Output

```
true
```

Question 5

```
System.out.println(!(10 > 5));
```

Output

```
false
```

Question 6

```
int age = 18;  
System.out.println(age >= 18 && age < 60);
```

Output

```
true
```

9. Cheat Sheet

Operator	Description	Example	Result
==	Equal to	5 == 5	true
!=	Not equal to	5 != 3	true
>	Greater than	10 > 5	true
<	Less than	3 < 2	false
>=	Greater than or equal	5 >= 5	true
<=	Less than or equal	4 <= 2	false
&&	Logical AND	true && false	false
	Logical OR	true false	true
!	Logical NOT	!true	false

Key Points to Remember

- Relational operators compare values and return a **boolean**.
- Logical operators combine multiple boolean conditions.
- && requires **all** conditions to be true.
- || requires **at least one** condition to be true.
- ! reverses a boolean value.
- Java uses **short-circuit evaluation** with && and ||.
- Use == for primitive value comparison and equals() for comparing String contents.